

Backlog del Sprint 1 — Scaps

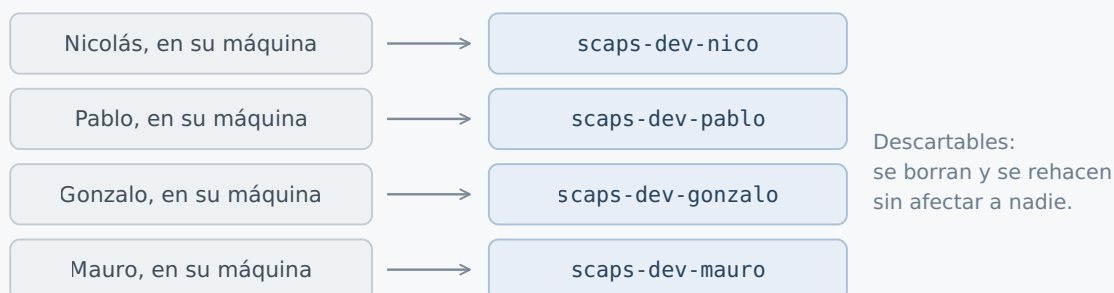
Autenticación y base del backend || base del frontend

Sprint 1 · 29 de junio al 12 de julio de 2026. Con el ERD y el contrato de la API cerrados en el Sprint 0, backend y frontend avanzan en paralelo desde el primer día contra una interfaz ya acordada. Al terminar el sprint tienen que estar arriba y desplegados: la autenticación con roles, la API completa de productos y el esqueleto navegable del frontend con login y registro funcionando contra la API real.

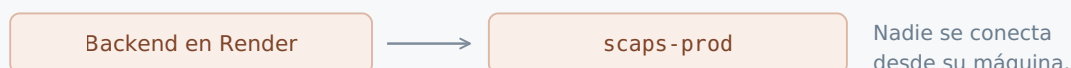
Cada tarea indica su área entre corchetes, sus labels, qué hacer, el criterio para darla por terminada y, cuando aplica, de qué depende. La forma de trabajo (columnas del tablero, flujo y reglas) está en `Sprint-0-Scaps.pdf` y no se repite acá.

Antes de arrancar: las bases de datos del proyecto. No se instala Postgres en ninguna máquina. Todas las bases corren en Neon, un servicio que las gestiona en la nube, y cada uno se conecta a la suya por internet con la cadena de conexión que tiene en su `.env`.

Desarrollo una base por integrante



Producción una sola, compartida



Son cinco bases en total. Las cuatro de **desarrollo** son una por integrante, y sirven para que cada uno pueda correr migraciones y romper lo que quiera sin afectar a los demás; son descartables, y si alguien deja la suya hecha un desastre, la borra, corre las migraciones y el seed de nuevo y sigue trabajando. La de **producción** es una sola y no se toca a mano: sus tablas se crean solas al desplegar el backend, y nadie apunta su entorno local a ella.

Están separadas en proyectos distintos y no en ramas del mismo proyecto por un motivo concreto: en el plan gratuito de Neon el cupo mensual de cómputo se mide por proyecto, y si se agota, ese proyecto se suspende hasta el mes siguiente. Con proyectos separados, por más que alguien queme su cupo desarrollando, la base de producción queda intacta.

Paso de setup, una vez por integrante (no es una tarea del tablero)

1. Crearte una cuenta gratuita en `neon.com`. No pide tarjeta de crédito. El alta te lleva directo a la pantalla de creación del primer proyecto: **ese proyecto va a ser tu base de desarrollo**, así que no busques un botón de "crear proyecto" después de registrarte.
2. Completar esa pantalla exactamente así:
 - **Project name:** `scaps-dev-nico`, `scaps-dev-pablo`, `scaps-dev-gonzalo` o `scaps-dev-mauro`, según quién seas.
 - **Postgres version:** 18. No la elijas por "la más reciente": si Neon suma una versión nueva más adelante, seguí eligiendo 18 igual, porque es la que ya tienen los demás y la que corre en producción. Si las versiones se desparejan, aparecen errores que solo le pasan a uno y son un dolor de cabeza rastrear.
 - **Region** (o *server*): la más cercana a Argentina. Si aparece São Paulo, esa; si no, la de Estados Unidos que ofrezca. Cuanto más lejos esté el servidor, más lenta va a ser cada consulta mientras programás. **Elegila con ganas: la región no se puede cambiar** una vez creado el proyecto.
 - **Database name:** dejá el que viene por defecto (suele ser `neondb`). Cambiarlo no aporta nada y la cadena de conexión ya lo incluye.
3. Copiar la *connection string* que te muestra al terminar: es la URL con usuario, contraseña y host que Prisma usa para conectarse. **Incluye tu contraseña**, así que no la pegues en el chat del equipo ni la subas al repositorio.
4. Pegarla en tu `.env` local como `DATABASE_URL`. El archivo `.env` está ignorado por git, así que tu cadena nunca se sube al repositorio.

To Do — se pueden tomar ya (no dependen de nada)

[infra] Crear la base de producción en Neon

infra

prioridad-alta

- **Tipo:** configuración en Neon (no es un PR).
- **Qué hacer:** crear en `neon.com` el proyecto `scaps-prod`, que va a ser la base de producción y la única compartida por el equipo. **Con Postgres 18 y la misma región que las bases de desarrollo:** si producción corre una versión distinta de la que usa el equipo, aparecen errores que no se reproducen en ninguna máquina. Además, dejar escrita en el README la receta para que cada integrante arme su propia base de desarrollo.
- **Dónde vive la cadena de conexión: en ningún archivo del repositorio**, ni en `docs/` ni en un `.env`. No hace falta guardarla en ninguna parte porque ya está guardada: vive en el panel de Neon, y de ahí se copia cuando hace falta. Su único destino es el panel de variables de entorno de Render, al desplegar el backend. Lo que el equipo tiene que poder alcanzar no es una copia de la cadena, sino **la cuenta de Neon**: quien pueda entrar ahí la recupera en diez segundos.
- **Hecho cuando:** existe el proyecto `scaps-prod` con Postgres 18, su cadena de conexión no aparece en ningún archivo del repositorio, y el README explica cómo cada uno crea su base de desarrollo.

Por qué esto no es exageración: la cadena de conexión incluye la contraseña de la base en texto plano. Si se commitea una vez, **queda en el historial de git para siempre**: borrar el archivo en un commit posterior no la saca, sigue estando en el commit viejo. Sacarla de verdad obliga a reescribir el historial y, aun así, a rotar la contraseña. **Y esta tarea cubre solo la base de producción:** las de desarrollo no son una tarea del tablero, son un paso de setup que cada integrante hace una vez por su cuenta, explicado al principio de este documento.

[setup] Esquema de Prisma y primera migración

setup prioridad-alta

- **Qué hacer:** traducir el ERD a `apps/api/prisma/schema.prisma`: las ocho tablas (Usuario, Direccion, Producto, ProductoImagen, Carrito, ItemCarrito, Orden, ItemOrden), el enum `EstadoOrden` y las relaciones. Respetar tres cosas: los **nombres de campo exactamente como están en el ERD** (`es_admin`, `creado_en`, `glb_url`), para que las respuestas de la API salgan con la forma del contrato sin traducir nada en el medio; los montos como `Decimal` y nunca `Float`; y las reglas de unicidad (`email`, `slug`, `usuario_id` en `Carrito`, y el par `carrito_id + producto_id` en `ItemCarrito`). Después correr `npx prisma migrate dev`, que genera el archivo de migración con el SQL y lo aplica contra tu base de desarrollo.
- **Hecho cuando:** la migración está commiteada, cualquiera del equipo puede correr `npx prisma migrate dev` y obtener las ocho tablas en su propia base, y `npx prisma studio` las muestra vacías y navegables.

Regla del equipo, sin excepciones: la estructura de la base se cambia **únicamente** editando `schema.prisma` y generando una migración. Nadie agrega ni modifica tablas o columnas desde una interfaz gráfica (el panel de Neon, pgAdmin, DBeaver). Un cambio hecho a mano no queda en el repo, el resto del equipo no lo tiene y la próxima migración puede pisarlo. Para *mirar y editar datos*, cualquier herramienta sirve; para cambiar la *estructura*, siempre el archivo.

[setup] Configuración base de NestJS

setup prioridad-alta

- **Qué hacer:** dejar el backend listo para que se le cuelguen los endpoints, configurando tres cosas en `main.ts`: (1) el **prefijo global** `/api`, para que todas las rutas queden bajo esa raíz — el `GET /health` que ya existe pasa a ser `GET /api/health`; (2) el **ValidationPipe global**, que es lo que hace que los DTOs validen automáticamente y devuelvan el formato de error del contrato; (3) el **CORS**, que define qué orígenes pueden llamar a la API — por ahora `http://localhost:5173` (el frontend en local), leído desde una variable de entorno para poder sumar los de Vercel después.
- **Hecho cuando:** `GET /api/health` responde `200`, un request con un campo inválido devuelve `400` con la forma `{ statusCode, message, error }` del contrato, y el frontend en local puede llamar a la API sin error de CORS.

[setup] Layout base del frontend: navbar y router

setup prioridad-alta

- **Qué hacer:** montar el esqueleto de navegación de la app con React Router: un **navbar persistente** (logo + enlaces) que se ve en todas las pantallas, y las rutas del MVP declaradas con una página vacía cada una, apenas con su nombre escrito: `/` (landing), `/catalogo`, `/producto/:slug`, `/carrito`, `/checkout`, `/mis-direcciones`, `/login`, `/registro`, `/admin`. También una pantalla de 404 para cualquier ruta que no exista. Todavía no hay contenido real en ninguna: la tarea es que la navegación funcione.
- **Hecho cuando:** se puede navegar entre todas las rutas, el navbar se mantiene visible en todas, una URL inventada cae en el 404 y recargar el navegador en cualquier ruta no rompe nada.

[visor-3d] Comprimir los `.glb` con Draco

visor-3d prioridad-alta

- **Qué hacer:** comprimir los cuatro modelos del Sprint 0 **antes de subirlos a ningún lado**. Draco es un compresor de geometría 3D: achica el peso del archivo reduciendo cómo se guardan los vértices, las normales y los índices de la malla, sin cambiar cómo se ve el modelo —siempre que no se lo use

de forma agresiva—. Se corre con `gltf-transform`, que no hace falta instalar de forma permanente: `npx @gltf-transform/cli draco entrada.glb salida.glb`.

- **Dónde quedan:** las dos versiones se guardan una al lado de la otra, en `docs/glb/originales/` y `docs/glb/comprimidos/`, junto al PDF de créditos que ya vive ahí. Así el listado de archivos muestra solo la diferencia de peso, sin tener que anotar números a mano, y queda el original por si más adelante hay que volver a comprimir con otros parámetros. **Sacarlos de `apps/web/public/models/`:** ahí estaban para que el POC los levantara en local, pero de ahora en más el `glb_url` apunta a R2, y todo lo que quede en `public/` se despliega a Vercel en cada build aunque nadie lo use.
- **Hecho cuando:** las dos carpetas existen con sus cuatro archivos cada una, los comprimidos pesan menos que sus originales, ya no queda ningún `.glb` en `public/`, y —el criterio que no hay que saltarse— **cada modelo comprimido abierto en el POC del visor se ve igual que el original**, sin deformaciones ni caras rotas.

No esperes milagros: Draco comprime la geometría, no las texturas, y en estos modelos buena parte del peso son las texturas. `cap-vanarsdale` pesa 7,3 MB con 93 mil triángulos: va a bajar bien, pero no de 7 MB a 1 MB. Si después de comprimir siguen pesando mucho, el problema son las texturas, y ese es otro tratamiento que va al sprint de performance. **Y una consecuencia que arrastra al Sprint 2:** un `.glb` comprimido con Draco no lo abre `useGLTF()` así nomás — el frontend necesita tener configurado el decodificador de Draco. Dejarlo anotado para la tarea del visor.

Backlog — esperan que se libere una dependencia

[setup] Cliente HTTP del frontend

setup

prioridad-alta

- **Depende de:** Layout base del frontend + Configuración base de NestJS.
- **Qué hacer:** crear un solo archivo por el que pasen **todas** las llamadas del frontend a la API. La idea es simple: en vez de que cada pantalla arme su propio pedido al servidor, todas le piden a una función compartida, y esa función se ocupa de los detalles que se repiten siempre.
- **El problema que resuelve:** sin este archivo, la pantalla de login tendría que escribir todo esto por su cuenta:

```
// pantalla de login
const res = await fetch('https://scaps-api.onrender.com/api/auth/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password })
})
if (!res.ok) {
  const err = await res.json() // { statusCode, message, error }
  throw new Error(Array.isArray(err.message) ? err.message[0] : err.message)
}
const { user, token } = await res.json()
```

Y la del catálogo casi lo mismo, y la del carrito también: la dirección del servidor repetida en quince archivos, el token repetido, y cada uno resolviendo los errores a su manera. Con el cliente armado, esa misma pantalla escribe una línea:

```
// pantalla de login
const { user, token } = await api.post('/auth/login', { email, password })
```

Todo lo demás ya está resuelto adentro.

- **Tiene que resolver cuatro cosas:**

- **La dirección del servidor.** Leerla de `VITE_API_URL` y no escribirla nunca a mano. En tu máquina apunta a `localhost`; desplegada apunta a Render. Si estuviera escrita en cada pantalla, pasar de una a otra sería editar quince archivos.
 - **El token de sesión.** Adjuntar solo el header `Authorization: Bearer <token>` cuando hay sesión iniciada, porque casi todos los endpoints lo piden.
 - **Los errores.** Acá hay una trampa que se lleva puesto a mucha gente: **fetch no falla cuando el servidor responde 401 o 409**. Solo rechaza si se cae la red. Ante un error devuelve normalmente, con `res.ok` en `false`, y `.json()` entrega el cuerpo del error sin protestar — así que si nadie mira `res.ok`, la pantalla sigue de largo con los datos en `undefined` y falla en silencio. Hay que chequearlo a mano, y además traducir el `{ statusCode, message, error }` de NestJS, donde `message` a veces es un texto y a veces una lista de varios. Resuelto una sola vez acá, ninguna pantalla puede olvidárselo.
 - **La espera.** Exponer si la llamada está en curso y, si pasa de unos 5 segundos, mostrar “despertando el servidor, esto puede tardar un minuto”. El backend en Render se duerme a los 15 minutos sin uso; escrito acá, ese mensaje aparece en toda la app sin repetirlo en ningún lado.
- **Hecho cuando:** una pantalla de prueba llama a `GET /api/health` usando el cliente y muestra el resultado; apagando el backend en local aparece un mensaje de error legible en lugar de una pantalla rota o un error crudo en la consola; y `VITE_API_URL` está documentada en `.env.example`.

Por qué va temprano en el sprint: todas las tareas de frontend que vienen después la usan —sesión, login, registro— y también el catálogo del Sprint 2. Si no está lista cuando esas arrancan, cada uno improvisa la suya y después hay que unificarlas, que es el doble de trabajo.

[infra] Bucket de Cloudflare R2 con los modelos 3D

infra prioridad-alta

- **Tipo:** configuración en Cloudflare + subida manual de archivos (no es un PR).
- **Depende de:** Comprimir los `.glb` con Draco.
- **Qué hacer:** crear el bucket de R2, habilitar el acceso público y subir a mano los cuatro `.glb` **ya comprimidos**. Configurar el **CORS del bucket** para que acepte pedidos desde el dominio del frontend: sin eso, el navegador bloquea la descarga del `.glb` por venir de otro origen.
- **Dónde se anotan las URLs:** al habilitar el acceso público, Cloudflare le asigna al bucket un subdominio con forma `pub-<32 caracteres>.r2.dev`, y la URL de cada archivo es esa base más el nombre del archivo. Así que no hay que anotar dieciséis URLs sueltas: alcanza con crear `docs/glb/URLs_R2.md` con **la URL base del bucket y el nombre de cada archivo subido**. Ese mismo archivo lo van a completar después las capturas de las imágenes, y es lo que va a leer quien escriba el seed.
- **Hecho cuando:** los cuatro `.glb` están subidos, `docs/glb/URLs_R2.md` existe con la URL base y los nombres, y —el punto importante— **uno de ellos se descarga desde una página web abierta en el navegador sin error de CORS en la consola**.

Por qué el CORS es el criterio central: es el único punto de esta tarea que no se detecta trabajando en local con archivos sueltos. Si queda mal configurado, el visor 3D va a funcionar perfecto en la máquina de todos y va a fallar recién al desplegarlo, en pleno Sprint 2. Se valida ahora y el riesgo desaparece. **Sobre el subdominio `r2.dev`:** Cloudflare avisa que no está pensado para producción, que tiene límite de peticiones y que no se cachea en el borde de la red. La alternativa es conectarle un dominio propio al bucket, que es lo que descartamos por costo. Para el tráfico de este proyecto no es un problema —el límite está en cientos de peticiones por segundo—, pero queda dicho para que nadie se sorprenda.

[visor-3d] Generar las imágenes de producto desde los .glb

visor-3d

prioridad-alta

- **Depende de:** Bucket de Cloudflare R2.
- **Qué hacer:** sacarle a cada modelo 3D entre **3 y 5 capturas** —las que sostenga sin verse mal— para usarlas como fotos del producto, en lugar de buscar fotos de bancos gratuitos: así la galería de la ficha y el visor 3D muestran el mismo objeto, y no dos gorras distintas.
- **Cómo: no a mano con la herramienta de recorte, sino exportando desde el propio canvas.** Three.js dibuja el modelo dentro de un elemento `<canvas>`, y un canvas sabe entregar sus píxeles como archivo. La idea es armar una página descartable en `apps/web` que cargue cada `.glb`, ubique la cámara en los ángulos definidos y exporte:

```
// <Canvas gl={{ preserveDrawingBuffer: true }} ...>
// ojo: sin esa opción, toBlob devuelve una imagen EN BLANCO,
// porque WebGL limpia el buffer apenas termina de dibujar

canvas.toBlob(
  blob => descargar(blob, `${producto}-${angulo}.webp`),
  'image/webp',
  0.9
)
```

Son unas cuarenta líneas y no suma ninguna herramienta: R3F y drei ya están desde el POC. La página se borra cuando la tarea termina. La ventaja no es ahorrar tiempo, es que las condiciones de abajo dejan de depender del pulso de quien captura: el lienzo mide 1200 porque se lo declarás, la cámara está a la misma distancia porque es el mismo número, y el fondo es idéntico en las veinte imágenes.

- **Qué tiene que quedar igual en todas:**
 - **fondo blanco**, de un solo tono parejo y sin transparencia: sin degradados, sin sombras de escenario y sin nada detrás del modelo. Es lo que hace la mayoría de los e-commerce, evita pelear con el canal alfa y hace que las cards no parezcan un collage;
 - la misma distancia de cámara, así las gorras se ven al mismo tamaño en todas las fotos;
 - la misma iluminación, para que un modelo no salga brillante y el otro apagado;
 - el mismo juego de ángulos de base en los cuatro productos: frente, tres cuartos y costado. Si el modelo lo sostiene, sumale atrás y desde arriba, hasta cinco;
 - **la imagen de portada tiene que ser el mismo ángulo en los cuatro productos** (por ejemplo, tres cuartos en todos). Esa es la que se ve en la grilla del catálogo, una al lado de la otra: si una gorra sale de frente y la de al lado de costado, la grilla queda desprolija. El resto de la galería sí puede variar, porque se mira de a una dentro de la ficha.
- **Formato:** todas las capturas en **1200 × 1200 píxeles**, cuadradas, exportadas como `.webp`, y **ninguna por encima de 200 KB**. Si alguna se pasa, bajale la calidad de exportación, no las dimensiones. El cuadrado es el estándar en catálogos: todas las cards quedan de la misma forma sin recortes raros, y el lienzo del visor se configura una sola vez. Los 1200 salen de que la imagen más grande —la principal de la ficha— va a ocupar unos 500 o 600 píxeles en pantalla, y las pantallas de alta densidad necesitan el doble de esa medida para no verse borrosas.
- **Después:** subirlas al bucket de R2 y sumar sus nombres a `docs/glb/URLs_R2.md`, junto a los de los modelos.
- **Hecho cuando:** cada producto tiene entre 3 y 5 imágenes subidas a R2 y anotadas en `docs/glb/URLs_R2.md`, **ninguna salió en blanco**, las cuatro portadas son del mismo ángulo, y puestas una al lado de la otra en tamaño de miniatura se leen como un catálogo coherente y no como cuatro capturas sueltas.

Revisá el low-poly antes de dar las capturas por buenas. Los cuatro modelos son muy dispares: `cap-danlyvostok` tiene unos 1.000 triángulos. Rotando en el visor eso se perdona, porque se entiende que es un modelo 3D; congelado como foto de producto en una card puede verse directamente tosco. Sacale una captura a ese primero y miralo en tamaño de miniatura antes de hacer el resto. Y si algún ángulo le queda feo, dejalo afuera: es preferible un producto con tres imágenes buenas que uno con cinco dispares. **Si la exportación se resiste**, el plan B es capturar a mano con la herramienta de recorte, pero entonces las condiciones de arriba pasan a depender del pulso y hay que revisar las veinte imágenes una por una antes de subirlas.

[setup] Seed de la base: admin y productos de demo

setup prioridad-alta

- **Depende de:** Esquema de Prisma + Generar las imágenes de producto.
- **Qué hacer:** escribir un seed (un script que carga datos iniciales en la base) en `apps/api/prisma/seed.ts`, con un script de npm que lo ejecute. Tiene que crear: un **usuario administrador** con `es_admin: true` y contraseña hashada, tomando email y clave de variables de entorno y nunca escritos en el código; y **los cuatro productos ficticios**, uno por modelo `.glb`, con nombre, descripción, precio, stock, su `glb_url` y sus imágenes apuntando a las URLs de R2 **que están listadas en** `docs/glb/URLs_R2.md`, con **exactamente una** imagen marcada como `es_principal` por producto y **exactamente uno** de los productos marcado como `destacado`. El script tiene que poder correrse dos veces seguidas sin duplicar filas ni explotar.
- **Hecho cuando:** corriendo el script sobre una base vacía queda cargada, corriéndolo de nuevo no rompe nada, y en `prisma studio` se ven el admin y los productos con sus imágenes.

Por qué los administradores no se crean por la API: el contrato define que `POST /auth/register` siempre da de alta usuarios con `es_admin: false`. Si existiera un endpoint para crear admins, sería el agujero de seguridad más grande de la app. Por eso el único camino es este script.

[auth] Registro, login y datos del usuario actual

auth prioridad-alta

- **Depende de:** Esquema de Prisma + Configuración base de NestJS.
- **Qué hacer:** implementar los tres endpoints de autenticación del contrato: `POST /auth/register`, `POST /auth/login` y `GET /auth/me`. Las contraseñas se guardan hashadas con **bcrypt** (hashar = convertir la clave en un texto del que no se puede volver atrás, así ni el equipo puede leer las contraseñas de los usuarios). El token es un **JWT** firmado con un secreto que vive en una variable de entorno, con **vencimiento de 2 horas**. Las entradas se validan con DTOs y `class-validator`: email con formato válido, contraseña de 8 caracteres como mínimo.
- **Hecho cuando:** los tres endpoints responden con las formas exactas del contrato; registrarse con un email ya existente devuelve `409`; una clave incorrecta devuelve `401`; `GET /auth/me` sin token devuelve `401`; y **el campo `password` no aparece en ninguna respuesta**.

Decisión tomada sobre el token: se guarda en `localStorage`, no en memoria. Guardarlo en memoria se pierde al recargar la página, y eso rompería la vuelta de Mercado Pago en el Sprint 3, que trae al usuario de regreso con una carga completa. El vencimiento corto de 2 horas es lo que acota el riesgo. La cookie `httpOnly` con `refresh token` queda anotada como deuda técnica para después del MVP.

[auth] Guard de rol de administrador

auth prioridad-alta

- **Depende de:** Registro, login y datos del usuario actual.

- **Qué hacer:** crear los dos *guards* de NestJS (un guard es una pieza que se ejecuta antes del endpoint y decide si el request pasa o se rechaza): uno que valide que el JWT sea válido y ponga el usuario a disposición del endpoint, y otro que además exija `es_admin: true`. Dejar un decorador cómodo para marcar rutas, del tipo `@Roles('admin')`, y aplicarlo a un endpoint de prueba para verificarlo.
- **Hecho cuando:** una ruta marcada como admin devuelve `401` sin token, `403` con el token de un usuario común y `200` con el del admin del seed.

[auth] Pruebas de humo de la autenticación

auth prioridad-alta

- **Depende de:** Guard de rol de administrador.
- **Qué hacer:** escribir pruebas automáticas end-to-end de los caminos de auth con **Jest y Supertest**, que ya vienen incluidos en el scaffold de NestJS (no se instala ninguna herramienta nueva). Cinco casos: registro exitoso; login exitoso; login con contraseña incorrecta; ruta protegida sin token; ruta de admin con un usuario común.
- **Contra qué base corren:** contra una base aparte, y no contra la de desarrollo de cada uno. El motivo no es cuidarla —son descartables, para eso están— sino que **una prueba automática tiene que dar el mismo resultado siempre**, y para eso necesita arrancar de un estado conocido. El caso concreto: la prueba de registro da de alta `test@test.com`; la segunda vez que la corrés ese email ya existe, el backend devuelve `409` y la prueba falla sin que nadie haya roto nada. Por eso las pruebas borran los datos que tocan antes de empezar — y si apuntaran a tu base de desarrollo, te la vaciarían en cada corrida y tendrías que rehacer el seed para seguir trabajando. Sumado a que, si dos personas apuntan a la misma base y corren las pruebas a la vez, se pisan y fallan las dos.
- **Cómo conseguirla:** no hace falta un proyecto nuevo de Neon: alcanza con **una rama del proyecto que ya tenés**. El plan gratuito permite 10 por proyecto, se crean en dos clics y salen con el esquema ya puesto, porque lo heredan de la rama padre. Su cadena de conexión va en una variable de entorno propia (por ejemplo `DATABASE_URL_TEST`), así nadie tiene que editar su `.env` cada vez que quiere correr las pruebas.
- **Hecho cuando:** `npm test` corre los cinco casos en verde desde el workspace de `api`, **correrlo dos veces seguidas da el mismo resultado**, la base de desarrollo de quien lo ejecuta queda intacta, y se documenta en el README cómo ejecutarlos.

Por qué ahora y no en el sprint de estabilización: el plan exige cubrir login como camino crítico. Escribir estas pruebas con el código recién hecho y fresco cuesta una fracción de lo que cuesta reconstruirlas en septiembre, cuando nadie se acuerde de por qué una decisión se tomó así.

[catalogo] Endpoints públicos de productos

catalogo prioridad-alta

- **Depende de:** Seed de la base.
- **Qué hacer:** implementar los tres endpoints públicos del contrato. `GET /products`: catálogo paginado, **solo productos activos**, en formato card (liviano, sin `glb_url` ni galería completa), con los parámetros `q`, `precio_min`, `precio_max`, `en_stock`, `sort`, `page` y `limit`. `GET /products/:slug`: la ficha completa, buscada por slug y no por id. `GET /products/featured`: el único producto destacado, con el objeto completo, y `404` si no hay ninguno.
- **Hecho cuando:** los tres devuelven exactamente las formas del contrato; los montos viajan como string decimal (`"15999.00"`) y no como número; el listado responde con el sobre `{ data, meta }`; `imagen_principal` siempre viene presente; y un producto con `activo: false` no aparece en el catálogo ni se puede abrir por su slug.

[catalogo] CRUD de productos para el administrador

catalogo

prioridad-alta

- **Depende de:** Guard de rol de administrador + Endpoints públicos de productos.
- **Qué hacer:** implementar los endpoints de administración de productos del contrato: `GET /admin/products` (incluye inactivos), `GET /admin/products/:id`, `POST /products`, `PATCH /products/:id` y `DELETE /products/:id`. Puntos finos: el `slug` se genera solo a partir del nombre y se garantiza único, pero **editar el nombre no cambia el slug** (la URL queda estable); el `DELETE` es **baja lógica**, o sea que setea `activo: false` y no borra la fila, para no romper las órdenes que ya compraron ese producto; y el `POST` crea el producto con sus imágenes en una sola transacción, exigiendo al menos una imagen.
- **Hecho cuando:** un usuario común recibe `403` en todos; se puede crear, editar y dar de baja un producto; validaciones de `precio > 0` y `stock ≥ 0` devuelven `400`; un producto dado de baja desaparece del catálogo público pero sigue apareciendo en `GET /admin/products`.

Los productos se crean con su modelo y sus imágenes. `glb_url` y el arreglo `imagenes` son **obligatorios**: un producto no se puede crear sin ellos. Lo que no existe todavía no es el dato, es el **canal para subir el archivo**. Y se pueden separar porque en el modelo de datos esos campos son *strings*: la base no guarda el `.glb` ni el `.webp`, guarda el texto de dónde están, y al backend le da igual cómo llegó esa URL a la mano de quien hace el request. **En este sprint la subida ya está hecha a mano** —los archivos se cargaron al bucket en las tareas del bucket y de las imágenes—, así que las URLs ya existen: se copian de `docs/glb/URLs_R2.md` y se pegan en el cuerpo del request. El endpoint recibe exactamente lo mismo que va a recibir en el Sprint 4. **Por eso `POST /uploads/model` y `POST /uploads/image` no entran acá:** existen para alimentar el formulario del admin, que recién aparece en el Sprint 4. Mientras ese formulario no exista, el CRUD se prueba con un cliente de API —Postman, Thunder Client o similar— escribiendo el JSON a mano, y en ese contexto tenerlos no cambiaría nada, porque igual habría que pegar una URL en el request. **Recomendación:** si esos requests se escriben en archivos `.http` (extensión REST Client de VS Code) en vez de dentro de una herramienta, quedan versionados en el repositorio y los otros tres los tienen con un `git pull`, revisados en el Pull Request como cualquier otro archivo, en lugar de vivir en la máquina de uno solo.

[catalogo] Galería de imágenes y producto destacado

catalogo

prioridad-media

- **Depende de:** CRUD de productos para el administrador.
- **Qué hacer:** implementar los endpoints que faltan para gestionar la galería (`POST`, `PATCH` y `DELETE` sobre `/products/:id/images`) y `PUT /products/featured` para elegir el destacado. Acá viven las reglas que la base no puede garantizar sola y que tiene que sostener el backend:
- **Reglas:**
 - exactamente una imagen por producto tiene `es_principal: true`; marcar una nueva desmarca la anterior;
 - no se puede borrar la última imagen de un producto (la relación es 1..*): para quitarla, se da de baja el producto entero;
 - hay un solo producto destacado a la vez; elegir uno nuevo reemplaza al anterior;
 - solo se puede destacar un producto activo;
 - al dar de baja un producto que estaba destacado, el backend le saca el flag automáticamente, para no dejar en la landing algo que ya no se vende.
- **Hecho cuando:** las cinco reglas se cumplen y cada una tiene su caso probado a mano contra la API.

Ojo con esta: no existe un `DELETE` para vaciar el destacado a propósito, y el backend **no auto-promueve** otro producto cuando el destacado se da de baja. Deja el lugar vacío y el dashboard le avisará al admin. Elegir el destacado es siempre una decisión deliberada, no algo que el sistema adivine.

[auth] Sesión en el frontend y rutas protegidas

auth prioridad-alta

- **Depende de:** Cliente HTTP del frontend + Registro, login y datos del usuario actual.
- **Qué hacer:** armar el store de sesión con **Zustand**: guardar el token en `localStorage` y el usuario en el estado, y al arrancar la app, si hay token guardado, llamar a `GET /auth/me` para rehidratar la sesión y confirmar que sigue vivo. Si devuelve `401` (token vencido), limpiar todo y mandar al login. Agregar dos envoltorios de ruta: uno que exija estar logueado y otro que además exija `es_admin`, redirigiendo a `/login` a quien no corresponda.
- **Hecho cuando:** después de iniciar sesión, **recargar el navegador mantiene la sesión**; entrar a `/admin` sin sesión redirige al login; entrar a `/admin` con un usuario común también; y cerrar sesión limpia el token y vuelve a bloquear las rutas protegidas.

[auth] Pantallas de login y registro

auth prioridad-alta

- **Depende de:** Sesión en el frontend y rutas protegidas.
- **Qué hacer:** construir las dos pantallas **contra la API real corriendo en tu máquina** (`npm run dev` del backend, apuntando a tu propia base de desarrollo), **no contra datos simulados en el frontend**. Registro pide email, contraseña, nombre y apellido — **no** pide teléfono, que se pide recién en el checkout para no sumar fricción al alta. Ambas validan del lado del cliente antes de enviar, muestran el error que devuelve el backend en un lugar visible (email ya registrado, credenciales incorrectas), deshabilitan el botón mientras el request está en curso para evitar el doble envío, y al terminar bien redirigen a la landing con la sesión iniciada.
- **Hecho cuando:** una persona puede registrarse desde cero, cerrar sesión, volver a entrar con esas credenciales, y ver un mensaje claro (no una pantalla rota) al equivocarse de contraseña o al intentar registrar un email repetido.

[setup] Landing: hero y llamado a la acción

setup prioridad-media

- **Depende de:** Layout base del frontend.
- **Qué hacer:** maquetar la landing con el navbar, un hero con el nombre de la marca, una bajada corta, una imagen de fondo o destacada, y el botón **Ver catálogo** que lleva a `/catalogo`. Con estilos de Tailwind, sin diseño fino.
- **Hecho cuando:** la landing se ve presentable en escritorio y en celular, y el botón navega al catálogo.

El visor 3D no entra acá: va en el Sprint 2, como dice el plan. Montarlo no es soltar el POC en la pantalla: necesita que `GET /products/featured` ya esté consumiéndose, el fallback obligatorio para que la landing nunca quede vacía ante un `404`, estados de carga mientras el `.glb` se descarga y se procesa, y manejo de error si el modelo no carga. En este sprint el hero lleva una imagen fija y listo.

[infra] Desplegar el backend en Render

infra prioridad-alta

- **Depende de:** Configuración base de NestJS + Esquema de Prisma + Crear la base de producción en Neon.

- **Qué hacer:** crear en Render un *web service* del plan gratuito conectado al repo, apuntando la `DATABASE_URL` al proyecto `scaps-prod` de Neon. La configuración tiene una trampa que conviene saber de antemano:
- **Configuración:**
 - **Root Directory:** dejarlo en la raíz del repo, NO en `apps/api`. Render ignora todo lo que quede fuera del root directory, y con npm workspaces el `package.json` con la clave `workspaces` y el `package-lock.json` viven en la raíz.
 - Build Command: `npm ci && npm run build -w apps/api`, con `prisma generate` incluido dentro del script `build` de `apps/api` (el cliente de Prisma se genera dentro de `node_modules`, que en el servidor se arma de cero en cada deploy).
 - Start Command: `node apps/api/dist/main.js`.
 - Build Filters: `apps/api/**`, para que un push que solo toca el frontend no dispare un redeploy inútil de la API.
 - Correr `prisma migrate deploy` contra la base de producción para crear las tablas, y ejecutar el seed una vez.
 - Cargar todas las variables de entorno en el panel de Render: `DATABASE_URL`, el secreto del JWT, las credenciales del admin del seed y el origen permitido para CORS.
- **Hecho cuando:** `GET /api/health` responde `200` en la URL pública de Render, `GET /products` devuelve los productos del seed, y la URL queda anotada en el README.

[infra] Desplegar el frontend en Vercel

infra

prioridad-alta

- **Depende de:** Desplegar el backend en Render + Cliente HTTP del frontend.
- **Qué hacer:** crear el proyecto en Vercel conectado al repo, con **Root Directory en `apps/web`** — acá sí, porque Vercel maneja workspaces de npm de forma nativa y detecta el gestor de paquetes por el lockfile de la raíz. Cargar `VITE_API_URL` apuntando a la URL de Render. Después volver al backend y sumar el origen de Vercel a la configuración de CORS.
- **Hecho cuando:** la app carga en la URL pública de Vercel, se puede registrar un usuario e iniciar sesión contra el backend desplegado, no hay errores de CORS en la consola, y **el preview de un Pull Request también funciona**.

La trampa del CORS con los previews: Vercel genera una URL distinta para cada Pull Request. Si el backend solo permite el dominio de producción, el que revise un PR va a abrir el preview y va a comer un error de CORS en todas las llamadas, sin entender por qué. Hay que permitir el patrón de los previews de Vercel, no una sola URL fija. **Y un segundo detalle:** Vite incrusta las variables `VITE_*` en el momento de compilar, no las lee en ejecución. Si algún día cambia la URL del backend, no alcanza con editar la variable en el panel: hay que redespargar el frontend.